

Multi-Agent Orchestrator

Planner/executor DAG, tools, self-correction

February 14, 2025

Headline Result

Self-correction lifts task success 41.9% -> 100%

Prepared for: Sai Veda
Publishing account: Nikeshk834

Project Context

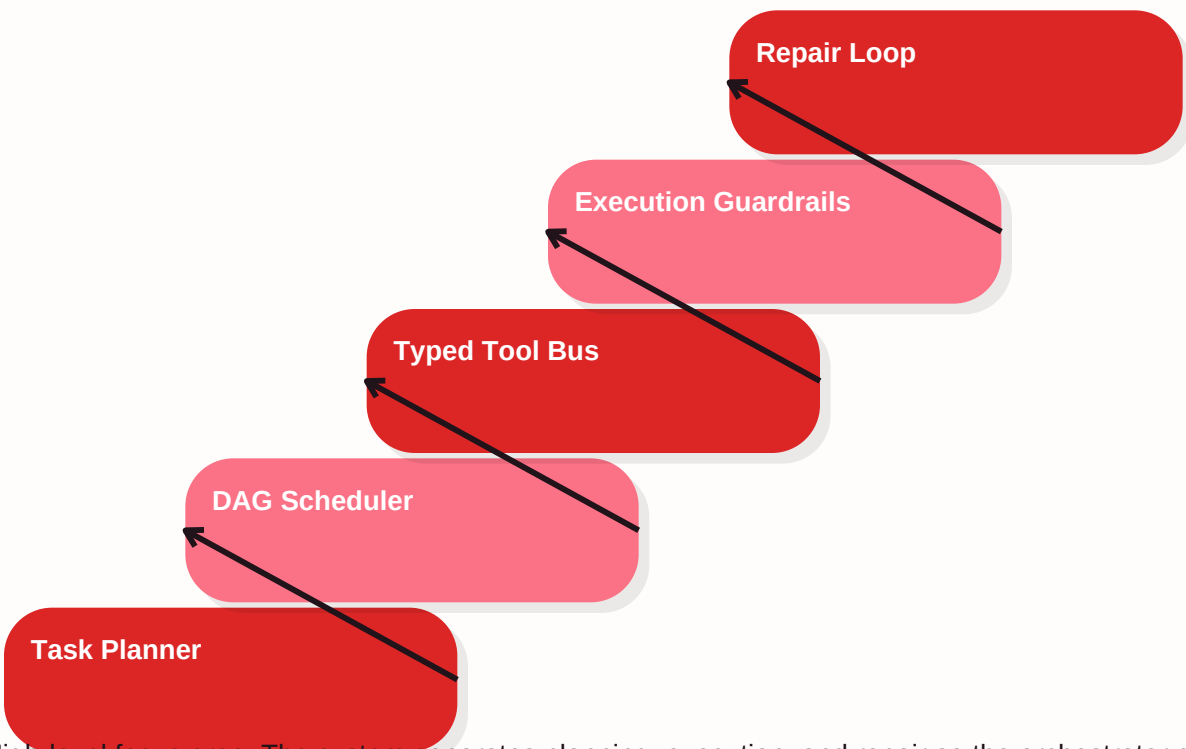
A from-scratch agentic framework - LangGraph / AutoGPT-class, but a clean, dependency-light design of its own. A planner decomposes a natural-language task into a DAG of tool calls; an executor runs that DAG respecting dependencies and running independent branches in parallel; guardrails validate inputs, tool arguments, and outputs; and a self-correction loop turns real execution feedback into a repaired plan and retries.

This PDF dossier is intentionally visual and interview-ready: it captures the business problem, the technical architecture, the validation evidence, and the committed repository artifacts in a format that reads like a delivered project brief.

Repository slug 03-multi-agent-orchestrator **Validation index** 69 tests

Architecture Storyboard

The system separates planning, execution, and repair so the orchestrator can recover from tool failures instead of collapsing.



High-level focus area: The system separates planning, execution, and repair so the orchestrator can recover from tool failures instead of collapsing.

Validation And Visual Evidence

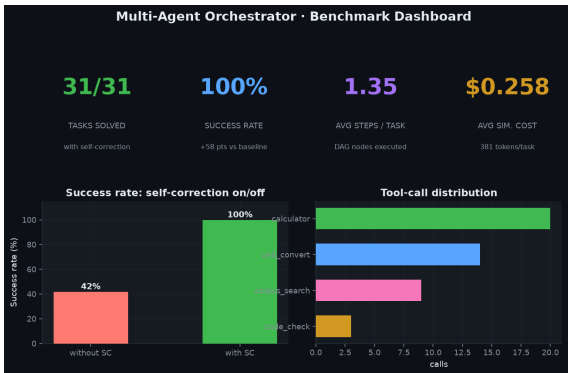
Self-correction lifts task success 41.9% -> 100%

69 passing tests

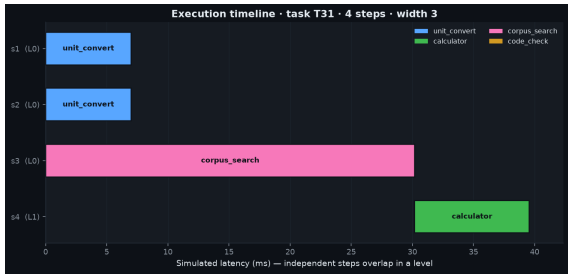
February 14, 2025

Selected benchmark table

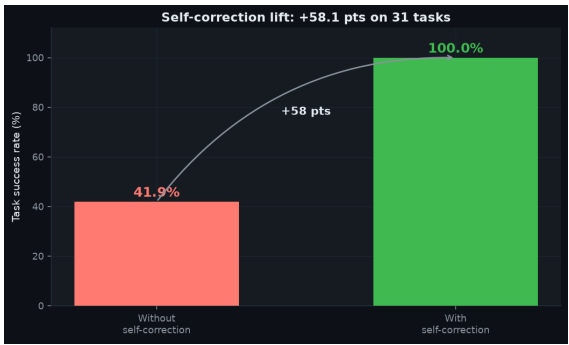
Condition	Success rate	Solved	Avg steps	Avg attempts
Without self-correction	41.9%	13/31	1.35	1.00
With self-correction	100.0%	31/31	1.35	1.58



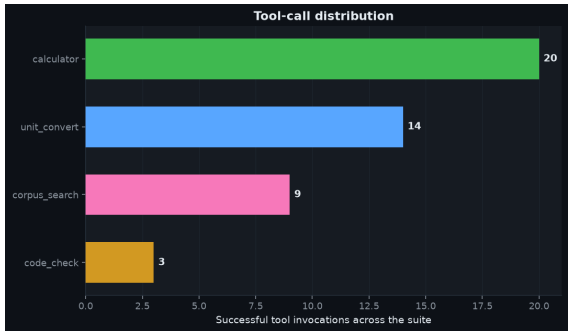
kpi_dashboard.png



execution_gantt.png



success_rate.png



tool_distribution.png

Repository Handoff

Prepared for: Sal Veda

Publishing account: Nikeshk834

Project date: February 14, 2025

Repository: 03-multi-agent-orchestrator

A from-scratch agentic framework - LangGraph / AutoGPT-class, but a clean, dependency-light design of its own. A planner decomposes a natural-language task into a DAG of tool calls; an executor runs that DAG respecting dependencies and running independent branches in parallel; guardrails validate inputs, tool arguments, and outputs; and a self-correction loop turns real execution feedback into a repaired plan and retries.

Committed deliverables

- README.md - product overview, quickstart, and measured headline numbers
- ARCHITECTURE.md - detailed design and scale trade-offs
- tests/ - automated validation suite
- benchmarks/ - measured run artifacts and result tables
- assets/ - generated dashboards, charts, and screenshots
- PROJECT_DOCUMENT.pdf - styled project brief for sharing and review

This document is intentionally more visual than the repository Markdown. It is meant to read like a polished project brief: business framing first, architecture second, proof third, and delivery scope last.